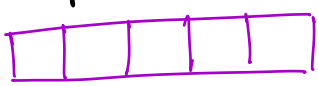
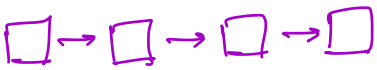
Today → Tree Basics

friday → subsequences & subsets

Arrays



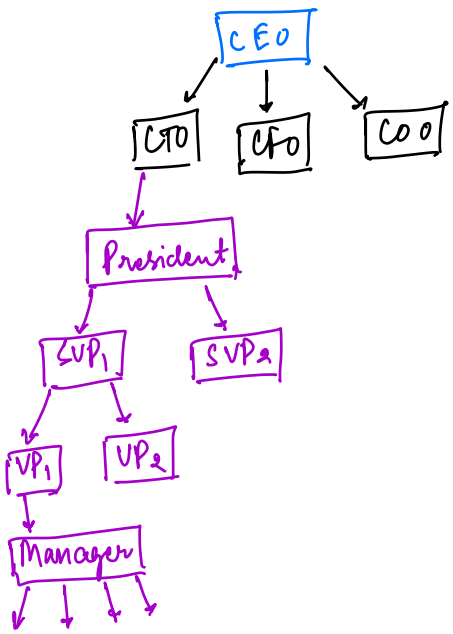
linked lists



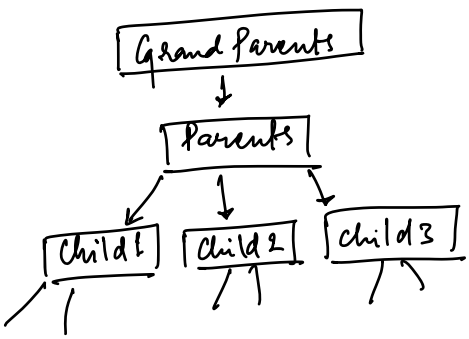
Linear DS till now.

Hierarchial Data

e.g. company's org structure

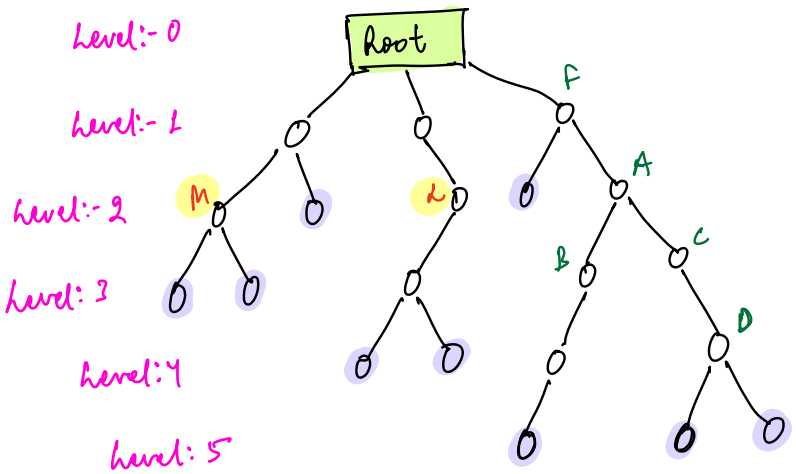


e.g. family Tree



Trees

Tree starts with a single node



0 → Leaf Nodes (No children)

* A is parent of B
* B and C are children of A
* A and F are ancestors of D
* B and C are siblings
* M and N belong to the same level

Naming conventions:-

→ Parent

→ Child

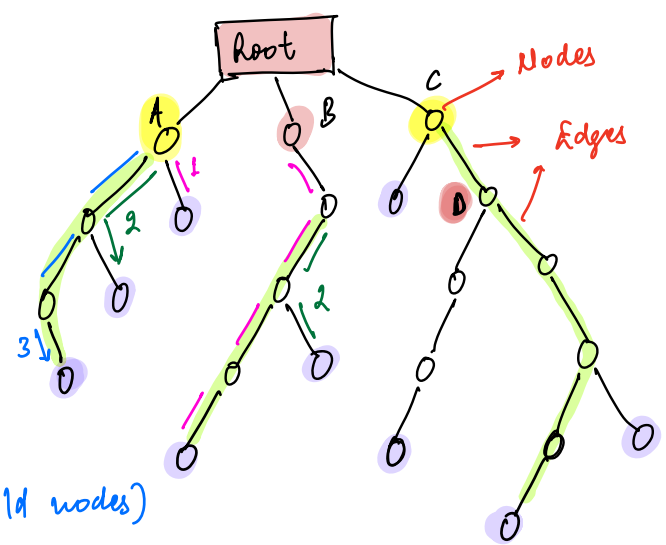
→ Ancestors / Descendants (D is descendant of A)

→ Siblings

→ Leaf Nodes

Height (Node):- length of the longest path from a node to any of it's descendant leaf nodes.

Path:- No. of edges.



Height (A) = 3
Height (B) = 4
Height (C) = 5

Height (root) = 1 + max (height of child nodes)
= 1 + 5 = 6

Height (D) = 1 + max (height of child nodes)

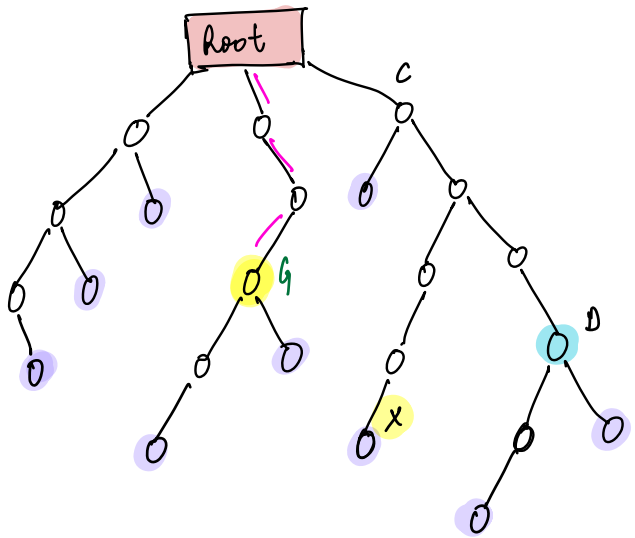
= 1 + max (2, 3) = 4

Height (Node) = 1 + max (height of all child Nodes) → Very important

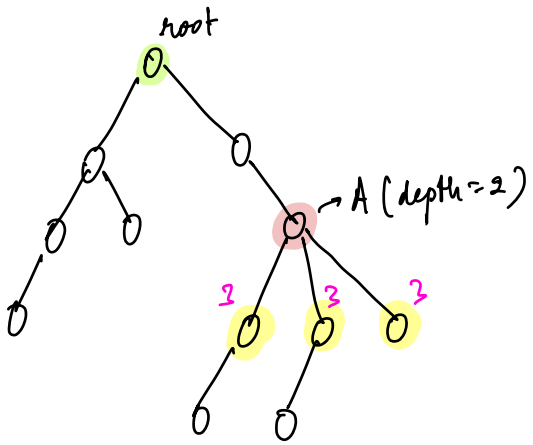
Note:- height of leaf Node = 0

* Depth (Node) → length of path from root node to given node

Depth (G) = 3
Depth (D) = 4
Depth (X) = 5



e.g.



depth (Node) = K
depth of all it's child Nodes = K+1
↳ important.

Ques what is height of a tree? = Height (Root Node)
Height (root) = height (tree) = 1 + max (3, 4, 5) = 6

Ques Depth (Root Node) = 0

Binary Tree → Discuss Today

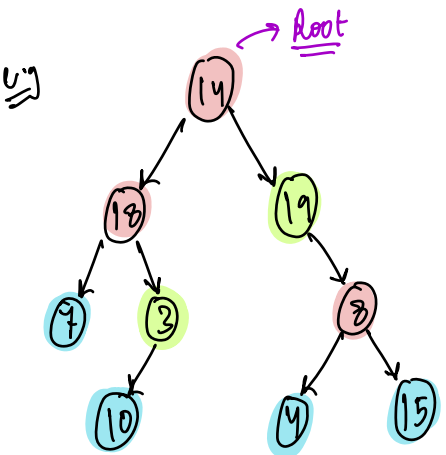
→ At max, every node can have only 2 child nodes

<= 2 {0, 1, 2}

N-Array Tree → Advance

→ Every node in your tree can have at max N child Nodes

<= N {0, 1, 2, 3 . . . N}



0 → No children

0 → 1 child Node

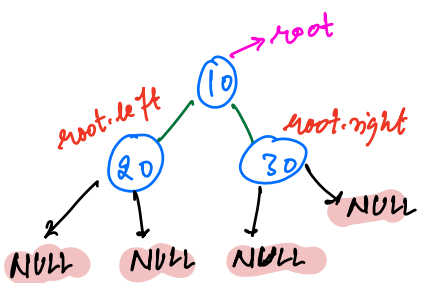
0 → 2 children

This is a Binary Tree

Structure of a Tree Node

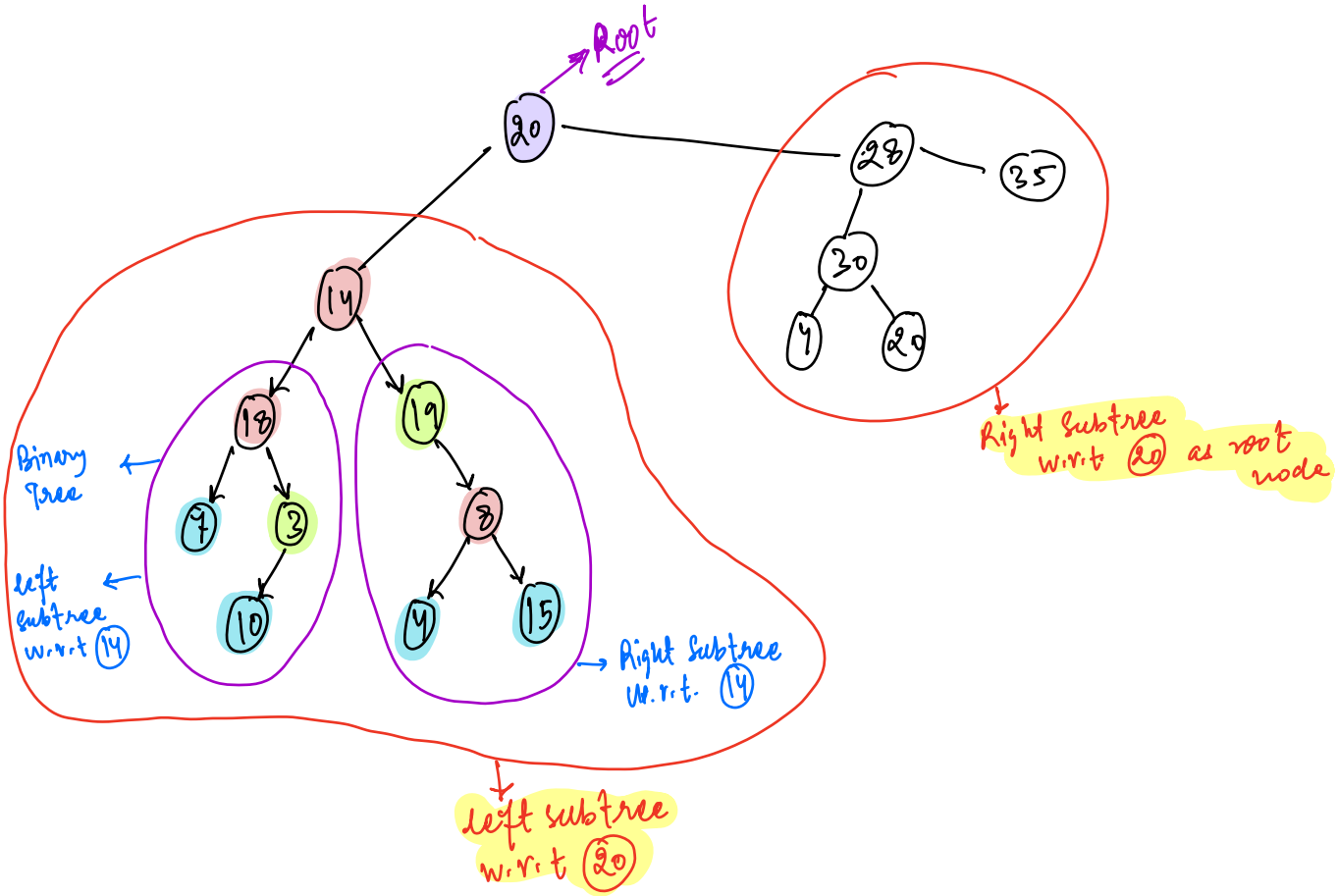
```
class Node {
    int data;
    Node left;
    Node right;
    Node(int x) {
        data = x;
        left = NULL;
        right = NULL;
    }
}
```

```
Node root = new Node(10);
root.left = new Node(20);
root.right = new Node(30);
```



Assumption for today's class

→ You are given a Binary Tree



Real World Applications of Trees

→ XML Parser
→ Decision Tree based Algorithms in ML
→ Databases use trees for indexing
→ Binary Trees are used in Computer Graphics
→ DOM in HTML
→ To store some possible move in a Chess Game
→ Used internally JVM to store Java objects.
→ Code compression Algorithm (.zip)
→ Forum websites (Quora)
→ Uber's system Design uses Quad Tree

Steps for Recursion:-

1) Assumption:- Decide what your func. should do and assume that it will do it.
2) Main logic:- Solving problems using subproblems
3) Base condition:- When code stops.

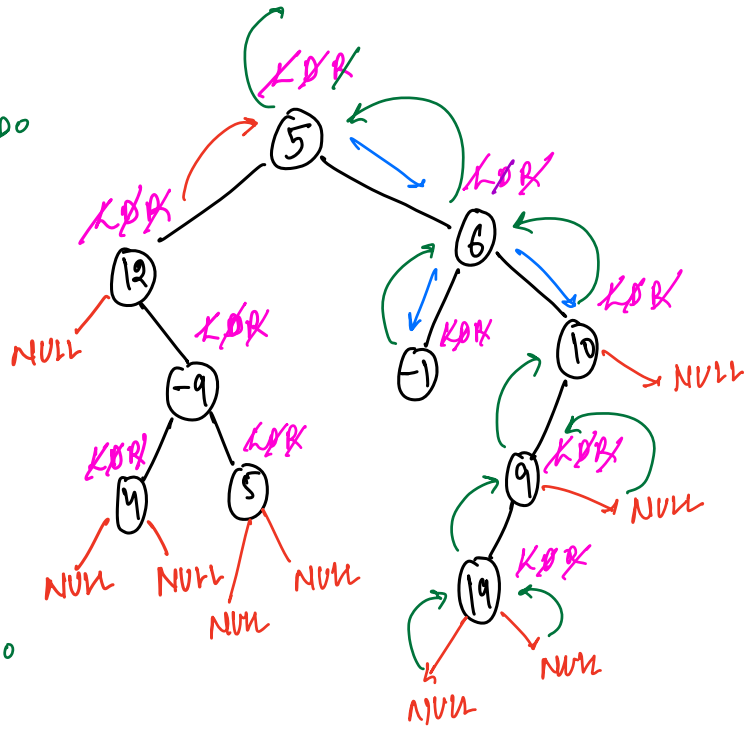
Tree Traversals

1) Pre-order :- [D L R] → TODO
(Data, Left, Right)

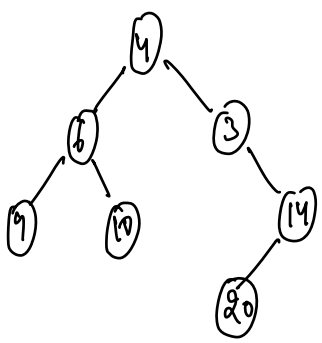
2) Inorder :- [L (D) R]
D → root node data

12, 4, -9, 5, 5, -1, 6, 19, 9, 10

3) Post order :- [L R D] → TODO



e.g.



inorder :- 9 6 10 4 3 20 14

preorder :- 4 6 9 10 3 14 20
D L R

postorder :- 9 10 6 20 14 3 4
L R D

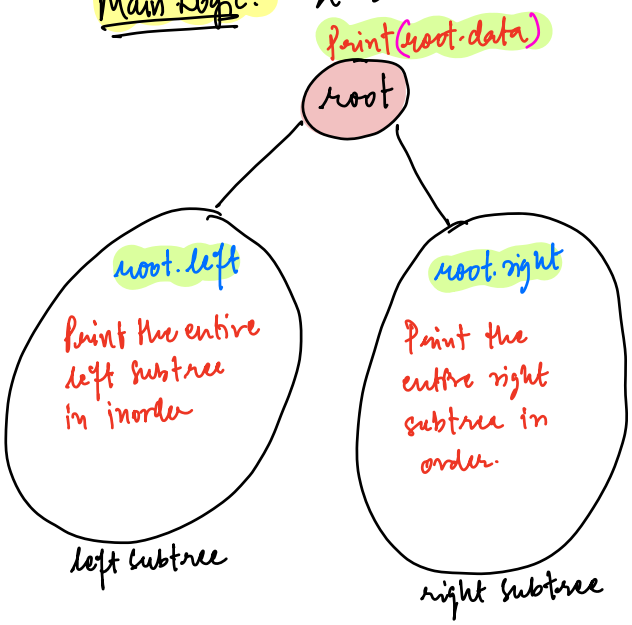
// inorder Traversal (Recursive)

```
void inorder (Node root) {
    ① if (root == NULL) return;
    ② inorder (root.left);
    ③ print (root.data);
    ④ inorder (root.right);
}
```

	L	D	R
inorder:-	1	2	3	4
preorder:-	1	3	2	4
postorder:-	1	2	4	3

Assumption:- Given root node, it will print the entire tree in inorder

Main logic:- L D R



Base condition

```
if (root == NULL) {
    return;
}
```

T.C = O(N)

① total no. of nodes in your BT

```
int size (Node root) {
    Ass:- Given a node, return size
    if (root = NULL) return 0;
    int l = size (root.left);
    int r = size (root.right);
    return l + r + 1;   → root node
}
```

② sum of all nodes of your BT

```
int sum (Node root) {
    Ass:- Given a node, returns sum of all nodes in that tree
    if (root = NULL) return 0;
    int ls = sum (root.left);
    int rs = sum (root.right);
    return ls + rs + root.data
}
```

TODO :- Implement height of a tree

↳ Discuss in friday's doubt session.

Merge Sort

Arrays.sort() X

Java